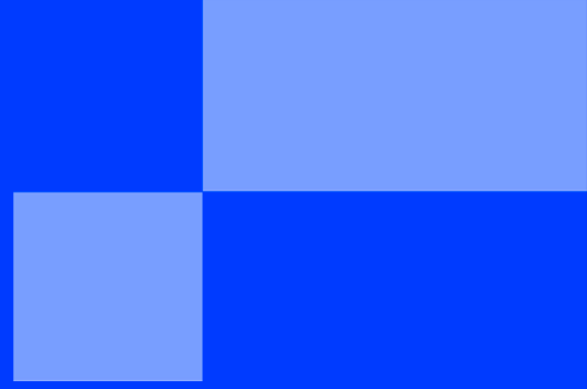




ATELIER PRÉDICTION - LA FIN DES CHANSONS LONGUES ?

< WINTER CAMP 2026 />

ATELIER PRÉDICTION - LA FIN DES CHANSONS LONGUES ?

Bienvenue dans cet atelier sur la prédiction ! Notre mission est d'utiliser un modèle de **régression linéaire** pour prédire la durée moyenne d'une chanson en 2030, et surtout de questionner les limites de cette prédiction.

Nous allons nous concentrer sur la période récente (après l'an 2000), où la tendance semble plus claire.

Étape 1 : Préparer les données pour l'analyse temporelle

Explication

Le fichier `tracks.csv` contient des millions de chansons. Pour analyser une tendance temporelle, il est plus pertinent de travailler sur des données agrégées. Nous allons donc commencer par **filtrer** les chansons pour ne garder que les plus récentes, puis nous allons **regrouper** (`groupby`) ces chansons par année pour calculer la durée moyenne pour chaque année.

Exemple

```
# Pour calculer une moyenne par groupe :  
df_evolution = df_recent.groupby('year')['duration_min'].mean().reset_index()
```

Expérimentation

✓ Dans un nouveau bloc de code :

- Chargez le fichier `tracks.csv` avec pandas et stockez-le dans une variable `df`.
- Filtrez pour ne garder que les chansons dont l'année est ≥ 2000 (ex. : `df_recent = df[df['year'] >= 2000]`).
- Agrégez : calculez la **durée moyenne par année** et stockez le résultat dans un DataFrame (ex. : `df_evolution`) avec des colonnes `year` et `duration_min`.
- Affichez un aperçu du tableau (ex. : `.head()`).



- ✓ `.reset_index()` après un `groupby(...).mean()` permet d'obtenir un DataFrame avec les noms de colonnes explicites au lieu d'un index multi-niveaux.

Étape 2 : Visualiser la tendance

Explication

Avant d'entraîner un modèle, il est essentiel de **visualiser** les données. Nous allons afficher un graphique pour voir l'évolution de la durée moyenne dans le temps. Un nuage de points avec une droite de régression superposée (ce que fait la fonction `regplot` de la bibliothèque **Seaborn**) est idéal pour voir si la tendance est "linéaire" (si elle ressemble à une droite).

Exemple

```
# Pour tracer année vs durée moyenne avec une droite de régression :
import seaborn as sns
import matplotlib.pyplot as plt

sns.regplot(x='year', y='duration_min', data=df_evolution)
plt.xlabel("Année")
plt.ylabel("Durée moyenne (minutes)")
plt.show()
```

Expérimentation

- ✓ **Dans un nouveau bloc de code :**
 - Tracez un graphique (line plot ou scatter + régression) avec **Axe X** : année, **Axe Y** : durée moyenne (en minutes).
 - Ajoutez des labels et un titre pour que le graphique soit compréhensible.
- ✓ **Bloc de Markdown :**
 - Voyez-vous une ligne qui descend ? Si oui, la régression linéaire peut bien capturer cette tendance.



- ✓ `sns.regplot()` affiche à la fois les points et la droite de régression.

Étape 3 : Entraîner le modèle de Machine Learning

Explication

Nous allons maintenant quantifier la tendance en créant un modèle mathématique qui suit la formule d'une droite : $y = a \cdot x + b$. **C'est un premier pas dans le monde du Machine Learning.**

- **X (appelé "Features")** : Ce sont les données que l'on donne au modèle pour qu'il apprenne. Ici, ce sera l'année.

- **y** (appelé "Target")** : C'est ce que l'on cherche à prédire. Ici, la durée moyenne.

En "entraînant" le modèle (avec la fonction `.fit()`), nous demandons à la bibliothèque **scikit-learn** de trouver les meilleures valeurs pour **a** (la pente) et **b**. La pente nous indiquera de combien de minutes les chansons raccourcissent en moyenne chaque année.

Exemple

```
from sklearn.linear_model import LinearRegression

X = df_evolution[['year']] # Toujours en 2D
y = df_evolution['duration_min']

model = LinearRegression()
model.fit(X, y)
print("Coefficient (minutes par an) :", model.coef_[0])
```

Expérimentation

✓ Dans un nouveau bloc de code :

- Définissez **x** avec la colonne des années (DataFrame 2D : `df_evolution[['year']]`).
- Définissez **y** avec la colonne des durées moyennes.
- Créez un modèle `LinearRegression()`, entraînez-le avec `.fit(X, y)`.
- Affichez le coefficient ; si vous le convertissez en secondes par an (`coef * 60`), interprétez en une phrase : « Chaque année, les chansons perdent environ X secondes. »



✓ **x** doit être un DataFrame (double crochet : `df_evolution[['year']]`).

Étape 4 : Faire nos prédictions

Explication

Maintenant que notre modèle est entraîné, nous pouvons l'utiliser comme un "oracle" pour **prédire** la durée moyenne pour des années qui ne sont pas dans nos données. C'est l'étape de prédiction (avec la fonction `.predict()`). C'est aussi le moment de faire preuve d'esprit critique et de se demander si le résultat a du sens.

Exemple

```
# Prédire la durée moyenne pour les années 2030 et 2050
model.predict([[2030], [2050]]) # format 2D
```

Expérimentation

✓ **Dans un nouveau bloc de code :**

- Prédisez la durée moyenne pour les années **2030** et **2050** avec `model.predict(...)` (format 2D, ex. : `[[2030], [2050]]`).
- Affichez les résultats de manière lisible (ex. : minutes et secondes).

✓ **Bloc de Markdown :**

- Le résultat pour 2030 vous semble-t-il réaliste ? Et pour 2050 ? En quoi le modèle « prolonge-t-il juste une ligne » sans comprendre le contexte ?



- ✓ Pour afficher une durée en minutes et secondes : `minutes = int(duree)`, `secondes = int((duree - minutes) * 60)`.

Bonus : La chanson de 0 seconde, c'est pour quand ?

Explication

Si on extrapole la droite jusqu'à une durée nulle, à quelle année arriverait-on ? Mathématiquement, on résout $0 = ax + b$, soit $x = -b / a$. Cela nous permet de calculer une date "théorique" de fin de la musique.

Expérimentation

- ✓ Calculez cette année à partir de `model.intercept_` (le `b`) et `model.coef_[0]` (le `a`).
- ✓ Discutez : ce résultat est-il crédible ? Cela illustre pourquoi il ne faut pas faire confiance à une extrapolation linéaire sans réfléchir au domaine (durée ≥ 0 , effets de saturation, etc.).

v1

{EPITECH}